MeKo MEDTECH

Git & GitHub Workshop

Arbeiten mit Git über GitHub

Christian Budde



Workshop-Agenda

Heute spielt ihr den **kompletten Weg einer Änderung** selbst durch. Am Beispiel-Repo „Teamwand“. Vorkenntnisse braucht ihr keine: Jeder Begriff wird eingeführt, wenn wir ihn brauchen.

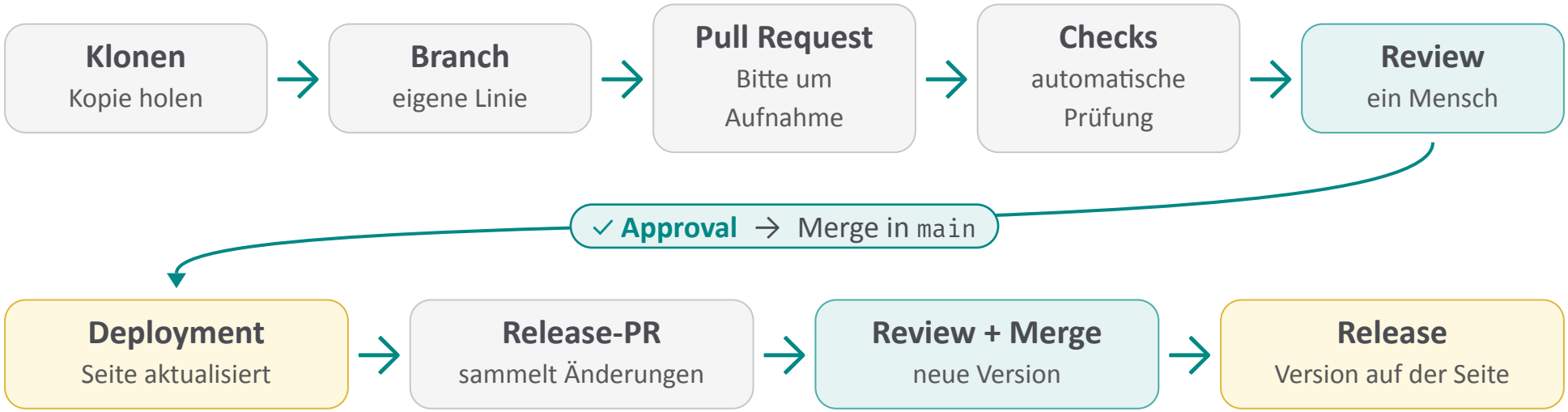
Eine Änderung pro Person: jede:r trägt sich in die Teamwand ein. Wir gehen den Weg **gemeinsam, Etappe für Etappe**.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Deployment | 7 · Release

Jede Übung zeigt die Befehle zum Selbertippen — mit Beispielprompts für einen KI-Assistenten.

Der Weg einer Änderung

Jede Änderung — egal wie klein — nimmt diesen Weg:



In echten Projekten gibt es dafür meist mindestens zwei Umgebungen: **Staging** (nach jedem Merge), **Prod** erst mit dem Release. Die Teamwand hier hat nur eine Seite für beides.

Niemand ändert main direkt — nicht ihr, nicht ein KI-Assistent, nicht die Admins.

Die Teamwand

Unser Übungsprojekt: eine Webseite mit einer Karte pro Person.

- `team.js` — die Einträge, eine Zeile pro Person
- `index.html` — die Seite, per Doppelklick öffnen
- `AGENTS.md` — die Regeln des Projekts, für Menschen **und** KI-Werkzeuge

Kein Build, kein Server — läuft direkt im Browser.

Repo

`MeKo-Christian/workshop-teamwand`

GitHub Pages — nach jedem Merge

`meko-christian.github.io/workshop-teamwand`

Release — neue Version im Footer

Release-PR mergen, dann Seite neu laden

Was ihr braucht

Werkzeuge

- Einen **GitHub-Account** — und die Einladung ins Repo angenommen
- **Git** installiert: `git --version` zeigt eine Version
- Einen **Editor**, z. B. VS Code
- Einen **Browser** für GitHub und die Seite

Einmalig: Git sagen, wer ihr seid

Jeder Commit trägt euren Namen. Im Terminal:

```
git config --global user.name "Anna Muster"
git config --global user.email "eure@mail.de"
```

Beim ersten **push** fragt Git im Browser nach dem GitHub-Login.

Optional: ein KI-Assistent

Wer einen nutzt (z. B. GitHub Copilot, Claude Code), kann ihm die Sätze von den Übungsfolien geben. Er tippt dann die Befehle — verstehen und prüfen müsst ihr trotzdem selbst.

Etappe 1: Klonen

Was liegt auf GitHub — und was holt ihr euch auf den Rechner?

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Deployment | 7 · Release

Zwei Kopien: Euer Rechner und GitHub

Euer Rechner

Eure eigene, vollständige Kopie.

- Arbeitsordner mit allen Dateien
- versteckter Ordner `.git` mit der ganzen History
- hier entstehen eure Commits — auch offline

Klonen



nur einmal pro Rechner

danach immer wieder:

Holen (pull)





Hochladen (push)

GitHub („origin“)

Das Original, für alle erreichbar.

- alle Dateien und die ganze History
- **nur hier:** Issues, Pull Requests, Reviews, CI
- „origin“ ist nur der Name, unter dem euer Rechner diese Kopie kennt

Klonen — einmal: kopiert alles und merkt sich „origin“.

Holen — vor jeder Arbeit: neue Commits von GitHub übernehmen.

Hochladen — nach dem Commit: eure Commits zu GitHub schicken.

Zwei Kopien — unabhängig voneinander

Nichts gleicht sich von selbst ab — nur wenn ihr **holt (pull)** oder **hochladet (push)**.

Ihr ändert etwas

Liegt **nur bei euch**. GitHub sieht es erst nach dem **push**.

Jemand anders ändert etwas

Liegt **nur auf GitHub**. Ihr seht es erst nach dem **pull**.

Beide ändern denselben Branch

Euer **push** wird abgelehnt: GitHub hat auf dem Branch Commits, die ihr noch nicht habt. Erst **pull** — Git führt beides zusammen. Neues auf `main` holt ihr dagegen eigens in euren Branch (Etappe 6).

Ihr löscht euren Ordner

GitHub bleibt unverändert. Neu klonen — weg ist nur, was ihr noch nicht per **push** hochgeladen habt.

Faustregel: vor der Arbeit pull, nach dem Commit push.

🛠️ Etappe 1: Klonen

Selbst machen (ca. 5 Minuten)

1. Klonen — im Terminal, in eurem Projektordner:
   ```
   git clone https://github.com/MeKo-Christian/workshop-teamwand
   cd workshop-teamwand
   ```
2. Die letzten fünf Commits ansehen: `git log --oneline -5`
3. Auf GitHub unter **Issues**: Welches Issue ist euch zugewiesen?

Selbst prüfen: `index.html` im Browser öffnen — die Seite läuft bei euch lokal.

Oder: sagt eurem KI-Assistenten

1. **„Klone MeKo-Christian/workshop-teamwand in meinen Projektordner."**
2. **„Zeig mir die letzten fünf Commits."**
3. **„Welche Issues sind mir auf GitHub zugewiesen?"**

Oder fragt: „Öffne die Teamwand lokal im Browser."

Etappe 2: Branch & Commit

Auf eigenem Branch arbeiten — vorerst nur auf eurem Rechner.

1 · Klonen | **2 · Branch & Commit** | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Deployment | 7 · Release

Was ist `main`?

`main` ist die **Hauptlinie** des Projekts. Beim Klonen bekommt ihr genau diesen Stand.



Nur ein Name

Der Name der Hauptlinie ist frei gewählt. Früher hat man es üblicherweise `master` genannt, heute meist `main`.

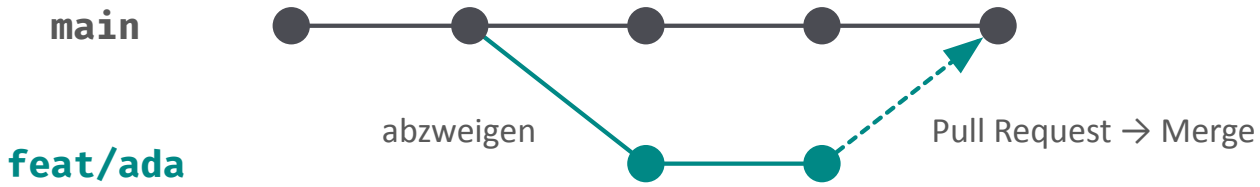
Geht auf die Seite

In diesem Repo landet jeder Merge in `main` automatisch auf der Seite — in echten Projekten meist auf Staging.

Neue Arbeit startet immer vom neuesten `main` — also erst pull.

Was ist ein Branch?

Ein Branch ist eine **eigene Linie von Commits**. Ihr zweigt von `main` ab, arbeitet in Ruhe — und bringt eure Änderung per Pull Request zurück.



Warum?

- `main` bleibt stabil — geht nach jedem Merge auf die Seite.
- Ihr stört niemanden, niemand stört euch.
- Ein Issue = ein Branch = ein PR: klein, gut zu reviewen.
- Schiefgegangen? Branch wegwerfen.

Gut zu wissen

- Kostet nichts: nur ein Name, der auf einen Commit zeigt.
- Name: Typ und Kurzbeschreibung, z. B. `feat/ada`.
- Entsteht lokal — GitHub sieht ihn erst nach dem push.
- Nach dem Merge wird er gelöscht.

Was ist ein Commit?

Ein Commit ist ein **gespeicherter Stand** eurer Dateien — mit Nachricht, Autor, Zeitpunkt und einer eindeutigen ID. Die History ist die Kette dieser Commits.

Ändern
im Editor

→

Vormerken
`git add`

→

Committen
`git commit`

→

Hochladen
`git push` · Etappe 3

Warum zwei Schritte?

Ihr entscheidet mit `git add`, **was** in den Commit kommt — nicht jede geänderte Datei gehört dazu.

Immer gut: `git status`

Zeigt, auf welchem Branch ihr seid, was geändert und was vorgemerkt ist.

Ein Commit entsteht nur auf eurem Rechner — GitHub sieht ihn erst nach dem push.

🛠️ Etappe 2: Branch & Commit

Selbst machen (ca. 10 Minuten)

1. Branch anlegen, Name wie im Issue:
   `git switch -c feat/7-teamwand-anna`
2. `team.js` im Editor öffnen und eure Zeile an der alphabetischen Stelle einfügen — Name, Emoji, Motto.
3. `index.html` im Browser neu laden: Ist eure Karte da?
4. Committen, noch nicht hochladen:
   `git add team.js`
   `git commit -m "feat: add anna to the team wall"`

Selbst prüfen: `git status` — alles committet? Und auf GitHub: Gibt es euren Branch dort schon?

Oder: sagt eurem KI-Assistenten

1. **„Ich will eine Änderung machen. Leg dafür einen Branch nach AGENTS.md an.“**
2. **„Erstelle eine persönliche Karte für mich — Name, Emoji, Motto.“**
3. **„Zeig mir das Ergebnis lokal im Browser.“**
4. **„Committe das.“**

Oder fragt: „Ist alles committet? Gibt es meinen Branch schon auf GitHub?“

Etappe 3: Push & Pull Request

Die Änderung geht zu GitHub — und ihr bittet um Aufnahme in `main`.

1 · Klonen | 2 · Branch & Commit | **3 · Push & PR** | 4 · Checks | 5 · Review | 6 · Merge & Deployment | 7 · Release

Was ist ein Pull Request?

Ein Pull Request (PR) ist die **Bitte**: „Nehmt meinen Branch in `main` auf.“ Der PR existiert nur auf GitHub — und bündelt alles, was dafür nötig ist:

Die Änderung

Reiter **Files changed**: jede Zeile, die anders ist als in `main`.

Die Prüfung

Automatische Checks und Kommentare von Menschen (und ggf. Bots).

Die Entscheidung

Freigabe durch einen Menschen, dann der Button **Merge**.

- Neue Commits auf dem Branch landen **automatisch im selben PR** — nach dem push.
- `Closes #7` im PR-Text schließt das Issue beim Merge.

Der PR-Titel wird zum Changelog

Der Titel des Pull Requests landet (in unserer Konfiguration) später **wörtlich** in den Release-Notes. Darum hat er bei uns eine feste Form:

```
typ: was sich ändert
```

Gut

```
feat: add anna to the
team wall
fix: keep long mottos
inside the card
```

Fällt durch die CI

```
Update team.js
fix stuff
```

Formal ok, sagt nichts

```
feat: changes
```

`feat` = neue Funktion · `fix` = Fehler behoben · `docs`, `chore`, `test` … = Sonstiges

🛠️ Etappe 3: Push & Pull Request

Selbst machen (ca. 10 Minuten)

1. Branch hochladen: `git push -u origin feat/7-teamwand-anna`
2. Auf GitHub erscheint **Compare & pull request** — klicken.
3. Titel `feat: add anna to the team wall`, Text `Closes #7` — dann **Create pull request**.

Selbst prüfen: Im PR unter **Files changed**: Titel verständlich? Nur eure eine Zeile geändert?

Oder: sagt eurem KI-Assistenten

1. **„Lade meinen Branch hoch.“**
2. **„Erstelle einen Pull Request mit Conventional-Commit-Titel, der mein Issue schließt.“**
3. **„Gib mir den Link zum PR.“**

Oder fragt: „Was ändert mein PR genau? Passt der Titel zu AGENTS.md?“

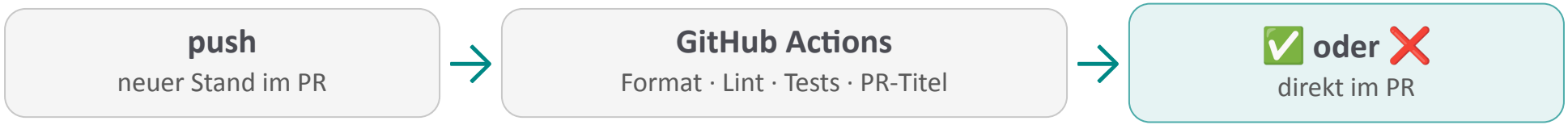
Etappe 4: Checks

GitHub prüft jeden Pull Request automatisch — bevor ein Mensch draufschaut.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | **4 · Checks** | 5 · Review | 6 · Merge & Deployment | 7 · Release

Was prüft die CI?

CI (Continuous Integration): Bei jedem push startet GitHub eine frische Maschine und prüft euren Branch — jedes Mal gleich, ohne Ausnahme.



Rot? Gut so.

Der Fehler ist gefunden, bevor ihn jemand sieht. **Details** öffnen, Log lesen, im selben Branch beheben, push — die Checks laufen neu. Regeln abschalten gilt nicht.

Kommentare im PR

Von Menschen — und, falls eingeschaltet, von einem KI-Reviewer wie GitHub Copilot. Jeden bewerten, beantworten und auflösen: berechtigt → beheben, sonst begründet ablehnen.

🛠️ Etappe 4: Checks

Selbst machen (ca. 10 Minuten)

1. Im PR unten: Checks abwarten — gelb läuft noch, dann grün oder rot.
2. Rot? **Details** öffnen, Log lesen, Datei im Editor korrigieren:
   `git commit -am "fix: sort my entry"` `git push`
3. Kommentare lesen und beantworten, dann **Resolve conversation**.

Selbst prüfen: Alle Checks grün, jeder Kommentar beantwortet und aufgelöst — vorher lässt sich nicht mergen.

Oder: sagt eurem KI-Assistenten

1. **„Sind die Checks in meinem PR grün?“**
2. **„Falls rot: Lies das Log und behebe den Fehler — ohne Regeln abzuschalten.“**
3. **„Fass die Kommentare in meinem PR zusammen. Was davon ist berechtigt?“**

Oder fragt: „Sind alle Checks grün und alle Kommentare aufgelöst?“

Etappe 5: Review

Das Vier-Augen-Prinzip: Ein Mensch gibt den PR frei.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | **5 · Review** | 6 · Merge & Deployment | 7 · Release

Gutes Review

Ihr als Reviewer:in

- Passt der PR zum Issue — und **nur** dazu?
- Selbst ansehen — gern lokal ausprobieren
- Ist der Titel ein guter Changelog-Eintrag?
- Konkret kommentieren: **was** und **warum**

Ein KI-Reviewer (falls eingeschaltet)

- Prüft gegen `AGENTS.md`, schnell und gründlich
- Kennt aber euer Ziel nicht — und kann irren
- **Gibt nichts frei** — das macht immer ein Mensch

🛠️ Etappe 5: Review

Selbst machen (ca. 10 Minuten)

1. Im eigenen PR rechts bei **Reviewers** die Person aus eurem Issue eintragen.
2. **Pull requests** → **Review requests**: den PR öffnen, den ihr prüfen sollt, und **Files changed** lesen.
3. Lokal ausprobieren: `git fetch` `git switch feat/8-teamwand-ben` — `index.html` neu laden.
4. **Review changes** → **Approve** oder **Request changes**, mit Kommentar.

Selbst prüfen: Freigeben klickt ihr **selbst** im Browser — und nur, was ihr selbst angesehen habt.

Oder: sagt eurem KI-Assistenten

1. **„Fordere für meinen PR ein Review von der Person an, die in meinem Issue steht.“**
2. **„Welche PRs soll ich reviewen?“**
3. **„Hol mir diesen PR auf den Rechner und fass zusammen: Passt er zum Issue und zu AGENTS.md?“**

Oder fragt: „Wurde mein PR freigegeben — und von wem?“

Etappe 6: Merge & Deployment

Ab in `main` — und automatisch auf die Seite.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | **6 · Merge & Deployment** | 7 · Release

Merge-Konflikt — keine Panik

Zwei PRs ändern **dieselbe Stelle**. Wer zuerst merged, gewinnt — der andere muss nachziehen.

```
<<<<<<< main
  { login: "anna", … },
=======
  { login: "ben", … },
>>>>>>> mein-branch
```

Was richtig ist, wisst ihr

Git kann nicht entscheiden, welche Zeile gilt. Hier: **beide**, alphabetisch.

Selbst: Im PR **Resolve conflicts** klicken, die Markierungszeilen löschen, beide Einträge behalten → **Mark as resolved** → **Commit merge**.

KI-Assistent: „Mein PR hat einen Konflikt mit main. Löse ihn so, dass beide Einträge erhalten bleiben, und erklär mir, was du gemacht hast."

🛠️ Etappe 6: Merge & Deployment

Selbst machen (ca. 15 Minuten)

1. Freigegeben und grün? Im PR **Squash and merge** — einer nach dem anderen.
2. Lokal auf den neuen Stand: `git switch main` `git pull`
3. Alten Branch löschen: `git branch -D feat/7-teamwand-anna`

Selbst prüfen: `git log --oneline -3` — steht euer Commit in main? Nach ein paar Minuten `meko-christian.github.io/workshop-teamwand` öffnen: Seid ihr auf der Seite zu sehen?

Oder: sagt eurem KI-Assistenten

1. **„Ist mein PR freigegeben und grün? Dann merge ihn.“**
2. **„Hol den neuesten Stand von main auf meinen Rechner.“**
3. **„Lösch meinen alten Branch lokal.“**

Oder fragt: „Ist mein Commit in main — und schon auf der Seite live?“

Etappe 7: Release

Alle Änderungen bekommen eine neue Versionsnummer.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Deployment | **7 · Release**

Release — Versionen aus PR-Titeln

Nach jedem Merge sammelt **release-please** alles in **einem** Release-PR:

- Changelog aus euren **PR-Titeln**
- neue Versionsnummer
- Merge des Release-PRs → **neue Version**

In echten Projekten geht genau dieser Stand dann nach Prod. Hier zeigt die Seite die neue Version im Footer.

chore(main): release 0.2.0

```
## 0.2.0

### Features
* add anna to the team wall (#9)
* add ben to the team wall (#11)
* add carla to the team wall (#12)
```

🛠️ Etappe 7: Release

Selbst machen (ca. 15 Minuten)

1. **Pull requests** → `chore(main): release …` öffnen und den Changelog lesen.
2. Gemeinsam: freigeben und mergen.
3. Reiter **Actions**: Ist `pages` durchgelaufen? Unter **Releases** steht die neue Version.

Selbst prüfen: `meko-christian.github.io/workshop-teamwand` neu laden — steht unten die neue Version?

Oder: sagt eurem KI-Assistenten

1. **„Zeig mir den offenen Release-PR und fass den Changelog zusammen.“**
2. **„Ist die Release-Pipeline nach dem Merge durchgelaufen?“**

Oder fragt: „Welche Version zeigt die Seite jetzt?“

Und wenn die neue Version kaputt ist?

Kein Hotfix per Hand: Änderung per Revert-PR zurücknehmen — gleicher Weg, auch unter Zeitdruck.

Das nehmt ihr mit

Der Ablauf

Klonen → Branch → Commit → Push → PR → Checks → Review → Merge → Deployment → Release

Jede Änderung, jedes Mal. Gerade die kleinen.

Weiterüben

Issues mit Label `bonus`: kleine Features mit Test — gleicher Weg.

Für euer eigenes Projekt: `main` schützen (Ruleset), CI als Pflicht, Squash-Merge, PR-Titel nach Conventional Commits.

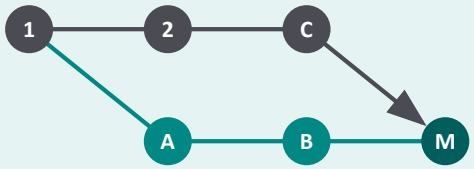
Ob ihr tippt oder ein KI-Assistent: Verantwortlich seid ihr — und das Review ist eure Unterschrift.

Anhang

Merge oder Rebase?

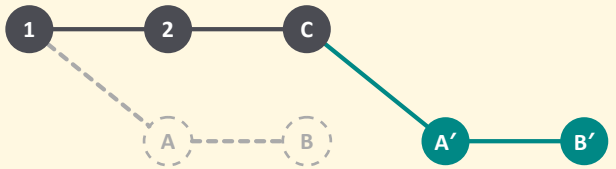
Euer Branch ist hinter `main` — inzwischen wurde `C` gemerged. Zwei Wege, ihn auf den neuesten Stand zu bringen:

Merge — zusammenführen



- `main` wird in euren Branch geholt, ein **Merge-Commit** `M` verbindet beide Linien.
- Eure Commits A, B bleiben, wie sie sind.
- History zeigt ehrlich, was wann passiert ist — mit Verzweigungen.

Rebase — neu aufsetzen



- Eure Commits werden **auf** das neue `main` neu abgespielt.
- Dabei entstehen **neue** Commits A′, B′ — die alten sind weg.
- History wird eine gerade Linie, als hättet ihr erst jetzt angefangen.

Der Code ist danach derselbe — nur die History sieht anders aus.

Merge oder Rebase — was heißt das für uns?

Auf GitHub: nur Squash-Merge

Der ganze PR wird **ein** Commit auf `main` — mit dem PR-Titel als Nachricht.

- `main` bleibt immer eine gerade Linie.
- Wie es in eurem Branch aussah, ist danach egal.

Im Branch: Merge ist der Standard

Euren Branch mit `main` abgleichen — beides geht:

- **Merge:** sicher, kein Umschreiben. Macht auch der Button „Update branch“.
- **Rebase:** schöner, aber danach braucht der push `--force-with-lease`.

Goldene Regel

Nie Commits rebasen, die andere schon haben — z. B. einen Branch, an dem zwei arbeiten. Wer seinen Stand schon geholt hat, bekommt sonst doppelte Commits. `main` ist deshalb gegen force push gesperrt.

Selbst: im PR **Update branch** — oder lokal `git fetch` `git merge origin/main` `git push`
KI-Assistent: **„Bring meinen Branch auf den Stand von main — per Merge, nicht per Rebase.“**